

Actividad 2: Manipulación y formateo de archivos: Formato FASTQ y FASTA

Objetivo

El propósito de esta actividad es desarrollar un flujo de trabajo bioinformático para el procesamiento y análisis de datos biológicos en los formatos **FASTQ** y **FASTA**. Se busca que el estudiante adquiera competencias para interactuar con la línea de comandos en un entorno Linux, manipular archivos de secuencias mediante distintas herramientas y desarrollar scripts en Shell para automatizar y resolver desafíos bioinformáticos comunes.

Detalles sobre la entrega

- La entrega se realizará a través del Campus VIU, mediante un único archivo en formato PDF, utilizando este documento como plantilla. El uso de cualquier otro formato conllevará una reducción del 20% en la calificación final.
- Es obligatorio trabajar exclusivamente en el entorno de AWS.
- Incluya el código empleado junto con capturas de pantalla que muestren su ejecución, incluyendo el *prompt* completo visible y en alta resolución. En caso de que la captura no sea legible o no incluya alguno de los elementos mencionados, la calificación de la pregunta se reducirá al 33% de su valor total.
- Proporcione explicaciones claras y estructuradas de los comandos utilizados, incluyendo sus opciones, órdenes y argumentos.
 - Si la explicación de los comandos es insuficiente, el valor de la pregunta se reducirá al 50%.
 - Si no se proporciona ningún tipo de explicación, la pregunta no será calificada.
 - En caso de que las instrucciones no sean comprensibles, se podrá requerir una explicación adicional en directo por parte del alumnado.
- No se permitirá la edición manual de archivos mediante editores de texto (como *pluma*, *vi* o *nano*) ni la integración de datos utilizando el comando *echo*.
 - La edición manual implica una calificación de 0 puntos.
- Para cada pregunta, deberá presentarse una única opción o método de resolución.
- No está permitido emplear otros lenguajes de programación, como Perl.

Parte I: Creación del directorio de trabajo.

1. Para inicializar este ejercicio, deberá crear un nuevo directorio de trabajo identificado como ***User_Project_Year_Publication***, donde:

- **User:** Corresponde al apellido del estudiante.
- **Project:** Es el nombre del proyecto hipotético en el que está trabajando.
- **Year:** Representa el año de la publicación o investigación en curso.
- **Publication:** Es el nombre de la revista donde planea publicar sus resultados.

Este directorio debe contener a su vez tres subdirectorios:

1. **data:** con las carpetas **raw** para datos originales y **processed** para datos procesados.
2. **code:** para los scripts generados.
3. **submission:** para almacenar el informe final.

Muestre una captura de pantalla donde se pueda observar la jerarquía de directorios creada.

Utilice el comando tree para ello. (0,25 pts)

Código:

```
mkdir galindo_k.pneumoniae_2026_nature
```

```
cd galindo_k.pneumoniae_2026_nature/
```

```
mkdir data code submission
```

```
cd data
```

```
mkdir raw processed
```

Explicación:

Se usa el comando mkdir para poder hacer los directorios solicitados, además para comprobar la jerarquía de directorios se usa el comando tree

Screenshot:

```
fgalindom@U-1K50FU0JT4NWU:~/Documents/programacion_shell$ mkdir galindo_k.pneumoniae_2026_nature
fgalindom@U-1K50FU0JT4NWU:~/Documents/programacion_shell$ cd galindo_k.pneumoniae_2026_nature/
fgalindom@U-1K50FU0JT4NWU:~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature$ mkdir data code submission
fgalindom@U-1K50FU0JT4NWU:~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature$ cd data
fgalindom@U-1K50FU0JT4NWU:~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/data$ mkdir raw processed
fgalindom@U-1K50FU0JT4NWU:~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/data$ cd ../../
fgalindom@U-1K50FU0JT4NWU:~/Documents/programacion_shell$ cd galindo_k.pneumoniae_2026_nature/
fgalindom@U-1K50FU0JT4NWU:~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature$ tree

.
├── code
├── data
│   ├── processed
│   └── raw
└── submission

5 directories, 0 files
```

Parte II: Obtención de datos


```
grep "^@SRR1984406" SRR1984406_1.fastq | cut -d" " -f3 | sort -V | head
```

```
grep "^@SRR1984406" SRR1984406_1.fastq | cut -d" " -f3 | sort -V | tail
```

```
grep "^@SRR1984406" SRR1984406_1.fastq | cut -d" " -f3 | sort -V | grep -c "length=75"
```

```
grep "^@SRR1984406" SRR1984406_1.fastq | cut -d" " -f3 | sort -V | grep -c "length=135"
```

Explicación:

Teniendo en cuenta la estructura del archivo analizada en el ejercicio anterior se determina que la longitud de cada secuencia se encuentra en la misma línea del identificador de secuencia, por lo que, para determinar las longitudes mínima y máxima se usa el comando **grep**, le pedimos que busque las líneas que inicien por **@SRR1984406**, este resultado lo redirigimos con el metacaracter pipe, usando el comando **cut** extraemos la columna donde se ubica **length**, para esto definimos el delimitador como espacio con la opción **-d** y además le pedimos que nos extraiga la columna 3 con la opción **-f** esto lo redirigimos al comando **sort** para que ordene estos datos teniendo en cuenta su naturaleza alfanumérica usando la opción **-V** por último, para ver el valor mínimo se usa el comando **head** donde se observa que la longitud mínima es 75 y con el comando **tail** se verifica que la longitud máxima es 135.

Con esto ya definido simplemente se adiciona al código anterior el comando **grep** con la opción **-c** que permita contar cuantas secuencias tienen una longitud de 75 y 135 buscando el texto de **length=75** o **length=135** en la salida ya obtenida por el código anterior dando como resultado 5 secuencias con longitud de 75 nucleótidos y 2006 secuencias con longitud de 135 nucleótidos.

Screenshot:

```

length=100
fgalindom@U-1K50FU0JT4NWU:~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/data/raw$ grep "^@SRR1984406" SRR1984406_1.fastq | cut -d " " -f3 | sort
-V | head
length=75
length=75
length=75
length=75
length=75
length=75
length=76
length=76
length=76
length=76
fgalindom@U-1K50FU0JT4NWU:~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/data/raw$ grep "^@SRR1984406" SRR1984406_1.fastq | cut -d " " -f3 | sort
-V | tail
length=135
length=135
length=135
length=135
length=135
length=135
length=135
length=135
length=135
length=135
length=135
length=135
length=135
fgalindom@U-1K50FU0JT4NWU:~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/data/raw$ grep "^@SRR1984406" SRR1984406_1.fastq | cut -d " " -f3 | sort
-V | grep -c "length=75"
6
fgalindom@U-1K50FU0JT4NWU:~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/data/raw$ grep "^@SRR1984406" SRR1984406_1.fastq | cut -d " " -f3 | sort
-V | grep -c "length=135"
2006

```

4. ¿Cuántas veces aparece el patrón “ATGATG” en las secuencias del archivo descargado? **(0,75 pts)**

Código:

```
grep -c "ATGATG" SRR1984406_1.fastq
```

Explicación:

Usando el comando **grep** con su opción **-c** permite buscar el patrón “ATGATG” en el archivo SRR1984406_1.fastq y contar cuantas veces aparece dando como resultado que aparece 773 veces.

Screenshot:

```

fgalindom@U-1K50FU0JT4NWU:~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/data/raw$ grep -c "ATGATG" SRR1984406_1.fastq
773

```

5. Transforme el archivo FASTQ a formato FASTA, extrayendo las líneas correspondientes al encabezado (*header*) y la secuencia de cada lectura. Guarde el archivo convertido como *all_sequences.fasta* en el directorio *data/processed*. Visualice las 4 primeras líneas del archivo resultante. **(2 pts)**

Código:

```
sed -n -e '1~4p' -e '2~4p' SRR1984406_1.fastq | head
```

```
sed -n -e '1~4p' -e '2~4p' SRR1984406_1.fastq > all_sequences.fasta | mv all_sequences.fasta
~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/data/processed
```

```
cd ..
```

```
tree
```

```
cd processed/
```

```
head -4 all_sequences.fasta
```

Explicación:

Usando el comando **sed** con su **opción -n** para que no imprima automáticamente las líneas que procese, luego usando el comando **-e** para que imprima las líneas que le pedimos explícitamente, **entre comillas** le indicamos que imprima primero la columna 1 y valla imprimiendo de 4 en 4 secuencialmente para esto se usa el metacaracter **~** y luego imprima la línea 2 de 4 en 4 igualmente. Este resultado se redirige usando el metacaracter **pipe** para comprobarlo usando el comando **head**.

Como vemos que funciona, el resultado del comando **sed** se redirige usando el metacaracter **>** a un archivo llamado **all_sequences.fasta** y luego de creado, usando el comando **mv** se mueve el archivo a la ruta de **~data/processed**. Para comprobar que quedo correctamente ubicado se usa el comando **tree** y por último me muevo al directorio donde quedo ubicado el archivo usando el comando **cd** y se verifica el contenido del archivo **all_sequences.fasta** usando el comando **head -4** para especificarle que queremos visualizar las primeras 4 líneas del archivo.

Screenshot:

```
fgalindom@U-1K50FU0JT4NWU:~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/data/raw$ sed -n -e '1~4p' -e '2~4p' SRR1984406_1.fastq | head
@SRR1984406.1 1 length=135
GACGACTGCCATCTGAACGTGTGGAATCAACGGAGCCACATCTGACTTCCAGTATCCATCCGAAGTTCTCCATTCAATAGTGAGGAATCTGACGACTGCCATCTGAACGTGTGGAATCAACGGAGCCACATCTGA
@SRR1984406.2 2 length=134
TTTGGGAATTTCTGTATCCATCCGAAGTTCTCCATTCAATAGTGAGGAATCTGACGACTGCCATCTGAACGTGTGGAATCAACGGAGCCACATCTGACTAGTCCCAGCATGAGCGACTCCACCGCCATTGGG
@SRR1984406.3 3 length=134
ATGCTGGGCACTAGTCAAAATGGGCTCCGTTGATTCACACGTTCCAGATGGCAATCGTCAGATTCTCTCACTATTGAATGGAGAATCTGGATGGATACTGGGTATCCGCGTTTCCAGTATCCATCCGAAGTCT
@SRR1984406.4 4 length=120
CCTCTGTAGTCAGCCCAATGGCGGTGGAGTCGTCATGCTGGGCACTAGTCAGATGGCTCCGTTGATTCACACGTTCCAGATGGCAGTCGTCAGATTCTCTCACTATTGAATGGGGATC
@SRR1984406.5 5 length=120
CCTCTGTAGTCAGCCCAATGGCGGTGGAGTCGTCATGCTGGGCACTAGTCAGGTGGCTCCGTTGATTCACACGTTCCAGATGGCAGTCGTCAGATTCTCTCACTATTGAATGGGGATC
fgalindom@U-1K50FU0JT4NWU:~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/data/raw$ sed -n -e '1~4p' -e '2~4p' SRR1984406_1.fastq > all_sequences
.fasta | mv all_sequences.fasta ~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/data/processed
fgalindom@U-1K50FU0JT4NWU:~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/data/raw$ cd ..
fgalindom@U-1K50FU0JT4NWU:~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/data$ tree
├── processed
├── all_sequences.fasta
├── raw
└── SRR1984406_1.fastq

2 directories, 2 files
fgalindom@U-1K50FU0JT4NWU:~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/data$ cd processed/
fgalindom@U-1K50FU0JT4NWU:~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/data/processed$ head -4 all_sequences.fasta
@SRR1984406.1 1 length=135
GACGACTGCCATCTGAACGTGTGGAATCAACGGAGCCACATCTGACTTCCAGTATCCATCCGAAGTTCTC
@SRR1984406.2 2 length=134
TTTGGGAATTTCTGTATCCATCCGAAGTTCTCCATTCAATAGTGAGGAATCTGACGACTGCCATCTGAACGTGTGGAATCAACGGAGCCACATCTGA
```

6. Después de la conversión, seleccione las primeras 5 lecturas de **all_sequences.fasta** y guarde cada una de ellas en un archivo individual con nombre **secuenciaX.fasta** (donde **X** representa el número de secuencia, del 1 al 5). Modifique la cabecera de cada archivo para que siga el formato **>Secuencia X** sin editar manualmente.

Como ejemplo, el archivo **secuencia1.fasta** contendrá:

>Secuencia 1

```
GACGACTGCCATCTGAACGTGTGGAATCAACGGAGCCACATCTGACTTCCAGTATCCATCCGAAGTTCTC
CATTCAATAGTGAGGAATCTGACGACTGCCATCTGAACGTGTGGAATCAACGGAGCCACATCTG
```

Visualice el contenido de todos los archivos creados y con el comando *ls* muestre la creación de éstos en el directorio *data/processed* (1,5 pts)

Código:

```
sed -n '1,2p' all_sequences.fasta | head
sed -n '1,2p' all_sequences.fasta | sed '1c >Secuencia 1' | head
sed -n '1,2p' all_sequences.fasta | sed '1c >Secuencia 1' > secuencia1.fasta
sed -n '3,4p' all_sequences.fasta | sed '1c >Secuencia 2' > secuencia2.fasta
sed -n '5,6p' all_sequences.fasta | sed '1c >Secuencia 3' > secuencia3.fasta
sed -n '7,8p' all_sequences.fasta | sed '1c >Secuencia 4' > secuencia4.fasta
sed -n '9,10p' all_sequences.fasta | sed '1c >Secuencia 5' > secuencia5.fasta
head secuencia1.fasta secuencia2.fasta secuencia3.fasta secuencia4.fasta secuencia5.fasta
ls -lh
```

Explicación:

Usando el comando *sed* con su opción *-n* para que no imprima automáticamente las líneas que procese, se extrajeron inicialmente las líneas 1 y 2 específicamente del archivo *all_sequences.fasta*, redirigiendo este resultado al comando *head* para visualizar la efectividad del código, como logré extraer las líneas de interés, luego en la siguiente línea de código ese resultado se redirigió y usando el comando *sed* con su opción *change* (*c*) se reemplaza la línea 1 por *>Secuencia 1* y evalué el código con el comando *head*, al verificar que el resultado está de acuerdo a lo solicitado en el ejercicio, en la siguiente línea de código, se dirige este resultado al archivo *secuencia1.fasta* usando el carácter mayor que (*>*), este código se repite para las siguientes 10 líneas del archivo *all_sequences.fasta*.

Usando el comando *head* se visualiza el contenido de los 5 archivos creados y por último usando el comando *ls* con sus opciones *-lh* que permiten visualizar los archivos en forma de lista y con lectura humana.

Screenshot:

```
fgalindom@U-1K50FU0JT4NMU:~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/data/processed$ sed -n '1,2p' all_sequences.fasta | head
@SRR1984406.1 1 length=135
GACGACTGCCATCTGAACGTGTGGAATCAACGGAGCCACATCTGACTTCCAGTATCCATCCGAAGTCTCCATTCAATAGTGAGGAATCTGACGACTGCCATCTGAACGTGTGGAATCAACGGAGCCACATCTGA
fgalindom@U-1K50FU0JT4NMU:~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/data/processed$ sed -n '1,2p' all_sequences.fasta | sed '1c >Secuencia
1' | head
>Secuencia 1
GACGACTGCCATCTGAACGTGTGGAATCAACGGAGCCACATCTGACTTCCAGTATCCATCCGAAGTCTCCATTCAATAGTGAGGAATCTGACGACTGCCATCTGAACGTGTGGAATCAACGGAGCCACATCTGA
fgalindom@U-1K50FU0JT4NMU:~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/data/processed$ sed -n '1,2p' all_sequences.fasta | sed '1c >Secuencia
1' > secuencia1.fasta
fgalindom@U-1K50FU0JT4NMU:~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/data/processed$ sed -n '3,4p' all_sequences.fasta | sed '1c >Secuencia
2' > secuencia2.fasta
fgalindom@U-1K50FU0JT4NMU:~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/data/processed$ sed -n '5,6p' all_sequences.fasta | sed '1c >Secuencia
3' > secuencia3.fasta
fgalindom@U-1K50FU0JT4NMU:~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/data/processed$ sed -n '7,8p' all_sequences.fasta | sed '1c >Secuencia
4' > secuencia4.fasta
fgalindom@U-1K50FU0JT4NMU:~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/data/processed$ sed -n '9,10p' all_sequences.fasta | sed '1c >Secuencia
5' > secuencia5.fasta
fgalindom@U-1K50FU0JT4NMU:~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/data/processed$ head secuencia1.fasta secuencia2.fasta secuencia3.fasta
secuencia4.fasta secuencia5.fasta
==> secuencia1.fasta <==
>Secuencia 1
GACGACTGCCATCTGAACGTGTGGAATCAACGGAGCCACATCTGACTTCCAGTATCCATCCGAAGTCTCCATTCAATAGTGAGGAATCTGACGACTGCCATCTGAACGTGTGGAATCAACGGAGCCACATCTGA
==> secuencia2.fasta <==
>Secuencia 2
TTTGGGAATTTCTGCTATCCATCCGAAGTCTCCATTCAATAGTGAGGAATCTGACGACTGCCATCTGAACGTGTGGAATCAACGGAGCCACATCTGACTAGTCCCCAGCATGAGCGACTCCACCGCCATTGGG
==> secuencia3.fasta <==
>Secuencia 3
ATGCTGGGCACTAGTCAAAATGGCTCGGTGATTCCACACGTTCCAGATGGCAATCGTCAGATTCTCTCACTATTGAATGGAGAACTTCGGATGGATAGTGGGTATCCCGTTTTCAGTATCCATCCGAAGTCT
==> secuencia4.fasta <==
>Secuencia 4
CCTCTGAGTCAGGCCAATGGCGGTGGAGTCGCTCATGCTGGGCACTAGTCAGATGTGGCTCGTTGATTCCACACGTTCCAGATGGCAGTCGTCAGATTCTCTCACTATTGAATGGGATC
==> secuencia5.fasta <==
>Secuencia 5
CCTCTGAGTCAGGCCAATGGCGGTGGAGTCGCTCATGCTGGGCACTAGTCAGGTGTGGCTCGTTGATTCCACACGTTCCAGATGGCAGTCGTCAGATTCTCTCACTATTGAATGGGATC
fgalindom@U-1K50FU0JT4NMU:~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/data/processed$ ls -lh
total 1.4M
-rw-r--r-- 1 fgalindom domain users 1.3M Jun 13 12:13 all_sequences.fasta
-rw-r--r-- 1 fgalindom domain users 149 Jun 13 16:51 secuencia1.fasta
-rw-r--r-- 1 fgalindom domain users 148 Jun 13 16:52 secuencia2.fasta
-rw-r--r-- 1 fgalindom domain users 148 Jun 13 16:53 secuencia3.fasta
-rw-r--r-- 1 fgalindom domain users 134 Jun 13 16:54 secuencia4.fasta
-rw-r--r-- 1 fgalindom domain users 134 Jun 13 16:54 secuencia5.fasta
```

7. Cree un script llamado *analyze_sequences.sh*, ubicado dentro del directorio *code*. Este script deberá procesar automáticamente cada uno de los cinco archivos FASTA generados anteriormente en el directorio *processed*, extrayendo y analizando cada una de las secuencias incluidas en ella.

Para cada secuencia, deberá generar un informe por la salida estándar que incluya:

- La longitud total de cada secuencia.
- La identificación del nucleótido inicial y final de cada secuencia.
- Porcentaje de contenido GC, redondeado a dos decimales. Atención: el redondeo no debe terminar en .00.
- Detección de los patrones "GGGG" y "TTCC":
 - Si se detecta alguno de ellos:
 - Informar al usuario de su presencia.
 - Mostrar únicamente el nombre de la secuencia donde aparecen.
 - Determinar cuántas veces aparece en el patrón en la secuencia.
 - Reemplazar únicamente el patrón GGGG por el patrón "Mutation" e imprimir la secuencia modificada.
 - En caso de no detectarse dichos patrones, notificar explícitamente su ausencia.

Adicione una captura de pantalla que muestre la estructura del script, su ejecución y los resultados obtenidos siguiendo las instrucciones de entrega previamente descritas. Además,

asegúrese de incluir explicaciones claras y ordenadas para cada uno de los comandos y pasos realizados en el script. **(3 pts)**

Código:

```
> fichero=$(ls  
~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/data/processed/secue  
ncia*)
```

Se emplea el comando ls para extraer todos los archivos que tengan la palabra secuencia en su nombre, esto se almacena en una lista llamada fichero. Esto permitirá recorrer cada archivo como un elemento de la lista más adelante.

```
> for fichero in "${fichero[@]}"  
do
```

Se usa el bucle for para ir recorriendo toda la lista llamada fichero, se define el iterador como fichero y usando el caracter @ se puede acceder a todos los elementos del array.

```
> sec=$(awk 'NR==2' "${fichero}")
```

Primero se extrae la secuencia del archivo fasta, para esto se usa el comando awk que con su variable interna NR permite extraer la segunda línea del iterador fichero y se almacena en la variable sec.

```
> leng=$(awk 'NR==2 {print length($0)}' "${fichero}")
```

Segundo, se calcula la longitud de la secuencia, para esto se usa nuevamente el comando awk, nuevamente se trabaja solo con la segunda línea del iterador fichero al usar la variable interna NR==2 y se imprime la longitud de caracteres usando la variable de bash length, esto se almacena en la variable leng.

```
> first_nu=$(awk 'NR==2 {print substr($0,1,1)}' "${fichero}")
```

```
> last_nu=$(awk 'NR==2 {print substr($0,length($0),1)}' "${fichero}")
```

Tercero, se busca el primer y el último nucleótido de la secuencia del iterador fichero, para esto, se usa el comando awk, se define la segunda línea del iterador fichero usando la variable interna NR==2, luego se usa las funciones de awk substr y print, la primera permite substraer el primero y el último nucleótido de la secuencia al definirse la variable como toda la línea, la posición de inicio, en el caso del primer nucleotido es la posición 1 y para el último nucleótido es la longitud de la secuencia que se calcula nuevamente con la variable de bash length, por último la longitud se define como 1 para solo extraer la posición de interes; la segunda funcion de print permite imprimir el resultado y se almacena en las variables first_nu y last_nu.

```
> cant_GC=$(echo "$sec" | grep -o '[GC]' | wc -l)
```

Cuarto, se calcula la cantidad de G y C en la secuencia, se usa primero echo para imprimir la variable sec y trabajar con la secuencia extraida anteriormente, este resultado se redirige usando el metacaracter pipe hacia el comando grep que con su opción -o busca y muestra el patrón dado por línea, el patrón de busqueda se coloca entre corchetes para buscar individualmente el contenido de G y C, esto se redirige hacia el comando word count con su opción -l que cuenta la cantidad de líneas que se obtuvieron con grep y se almacena en la variable cant_GC.

```
> percent_GC=$(echo "scale=10; $cant_GC/$leng * 100" | bc)
```

Quinto, se calcula el porcentaje de GC usando el comando bc, primero usando el comando echo se define una cantidad de 10 decimales con el comando scale, empleando las variables cant_GC y leng se realiza la operación total de GC/total de nucleótidos y se multiplica por 100, este resultado se redirige al comando bc y la operación total se almacena en la variable percent_GC.

```
> percent_GC_R=$(printf "%.2f" "$percent_GC")
```

Como se solicita que el decimal no sea .00, se crea la variable percent_GC_R la cual simplemente se transforma el resultado de percent_GC a 2 decimales usando la función printf definiendo la cantidad de decimales con el parámetro %.2f.

```
> gggg_count=$(echo "$sec" | grep -no "GGGG" | wc -l)
```

```
> ttcc_count=$(echo "$sec" | grep -no "TTCC" | wc -l)
```

Sexto, se buscan los patrones GGGG y TTCC, para esto primero se usa el comando echo para imprimir la secuencia a trabajar, luego se redirige con el metacaracter pipe hacia el comando grep que con sus opciones -no muestra el patrón buscado junto con la línea donde se encuentra, en este caso el patrón solo se coloca entre comillas dobles para que la búsqueda sea exacta y se redirige hacia el comando word count con su opción -l que permite contar la cantidad de líneas que se generan con el comando grep y así conocer cuantas veces aparece el patrón buscado, esto se almacena en las variables gggg_count y ttcc_count respectivamente.

```
> name_sec=$(awk 'NR==1' $fichero)
```

Septimo, se extrae el nombre de la secuencia que se usará más adelante, por lo que se almacena en la variable name_sec, para eso se usa el comando awk donde con la variable interna NR se definió la primera línea del iterador fichero donde se encuentra el nombre de la secuencia.

```
> echo "Longitud de Secuencia: $leng"
```

```
> echo "Primer Nucleótido: $first_nu"
```

Para el informe se uso el comando echo y aprovechando la ampliación de variables se adiciono el texto respectivo a cada item acompañado de la variable donde se encontraba el dato almacenado. En este caso solo se muestran 2 líneas de código, como ejemplo, ya que para todo el informe se uso la misma estructura, ademas de unas líneas de código que sirvio como verificación y que no se muestran en el informe final, pero se señalan como comandos de verificación, esto permitió ayudar a identificar si las variables estaban generando el resultado esperado antes de iniciar la sentencia de control de flujo.

```
> if (($gggg_count > 0))
```

```
then
```

```
echo "Presencia de patron GGGG detectada"
```

```
echo "Número de ocurrencias: $gggg_count"
```

```
echo "Nombre de secuencia: $name_sec"
```

```
mutation_g=$(echo "$sec" | sed 's/GGGG/"Mutation"/g')
```

```
echo "Secuencia modificada: $mutation_g"
```

```

else
echo "patron GGGG no detectado"
fi
> if (($ttcc_count > 0))
    then
        echo "Presencia de patron TTCC detectada"
        echo "Número de ocurrencias: $ttcc_count"
        echo "Nombre de secuencia: $name_sec"
    else
        echo "patron TTCC no detectado"
    fi
done

```

Octavo, para poder reportar si se detectaba los patrones GGGG y TTCC se uso una condición if compuesta. Para la comparación se uso las variables gggg_count y ttcc_count respectivamente comparando si el resultado era mayor que cero, si esta condición se cumplía se ejecutaba el comando echo para mostrar por pantalla que el patrón se encontraba en la secuencia, se mostraba el número de ocurrencias de ese patrón en la secuencia usando el resultado de la variable gggg_count o ttcc_count según correspondiera y mostrando el nombre de la secuencia con el resultado de la variable name_sec. Para el caso particular del patrón GGGG se incluyo una variable llamada mutation_g en la cual por medio de un comando echo se imprimía la secuencia, este resultado se redirigió al comando sed usando el metacatacter pipe y usando la opción substitute (s) se buscó el patrón GGGG y se reemplazó por la palabra mutation entre comillas dobles y usando la opción global (g) hacía que se reemplaza en todas las coincidencias de la línea. Una vez hecho esto se imprimía la secuencia modificada usando el comando echo y el resultado de la variable mutation_g. En dado caso que la variable gggg_count o ttcc_count según correspondiera arrojará un resultado de cero, se ejecutaba el comando echo donde se informaba que el patrón no había sido detectado.

Por último, se finaliza el bucle for al usar done.

Screenshot:

```

fgalindom@U-1K50FUOJT4NWU: ~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/code
#!/usr/bin/env bash
#
# Búsqueda de archivos a procesar
fichero=$(ls ~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/data/processed/secuencia*)
# Ejecución de comandos
for fichero in "${fichero[@]}"
do
    sec=$(awk 'NR==2' "$fichero")
    leng=$(awk 'NR==2 {print length($0)}' "$fichero")
    first_nu=$(awk 'NR==2 {print substr($0,1,1)}' "$fichero")
    last_nu=$(awk 'NR==2 {print substr($0,length($0),1)}' "$fichero")
    cant_gc=$(echo "$sec" | grep -o '[GC]' | wc -l)
    percent_gc=$(echo "scale=10; $cant_gc/$leng * 100" | bc)
    percent_gc_R=$(printf "%.2f" "$percent_gc")
    gggg_count=$(echo "$sec" | grep -no "GGGG" | wc -l)
    ttcc_count=$(echo "$sec" | grep -no "TTCC" | wc -l)
    name_sec=$(awk 'NR==1' "$fichero")

    # Extraer secuencias de cada archivo
    # Cálculo de longitud de la secuencia
    # Búsqueda de primer nucleótido
    # Búsqueda de último nucleótido
    # Cálculo de cantidad de GC por secuencia
    # Cálculo de % GC
    # Redondeo a 2 cifras
    # Búsqueda de patrón GGGG y conteo de veces que aparece en la secuencia
    # Búsqueda de patrón TTCC y conteo de veces que aparece en la secuencia
    # Extracción de nombre de secuencia

    echo "
    INFORME $name_sec
    "

    # echo "Archivo Procesado: $fichero"
    echo "Longitud de Secuencia: $leng"
    echo "Primer Nucleótido: $first_nu"
    echo "Ultimo Nucleótido: $last_nu"
    # echo "secuencia: $sec"
    echo "Porcentaje GC (%GC): $percent_gc_R %"
    # echo "GGGG: $gggg_count"
    # echo "TTCC: $ttcc_count"
    # echo "nombre secuencia: $name_sec"

    Comando de verificación
    Comando de verificación
    Comando de verificación
    Comando de verificación
    Comando de verificación

    echo "
    echo "

    DETECCIÓN DE PATRÓN GGGG
    -----
    "

    if (($gggg_count > 0))
    then
        # Verificación presencia de patrón GGGG en la secuencia
        echo "Presencia de patrón GGGG detectada"
        echo "Número de ocurrencias: $gggg_count"
        echo "Nombre de secuencia: $name_sec"
        mutation_g=$(echo "$sec" | sed 's/GGGG/"Mutation"/g')
        echo "Secuencia modificada: $mutation_g"
        # Reemplazo del patrón GGGG por "Mutation"
    else
        echo "patrón GGGG no detectado"
    fi

    echo "

    DETECCIÓN DE PATRÓN TTCC
    -----
    "

    if (($ttcc_count > 0))
    then
        # Verificación presencia de patrón GGGG en la secuencia
        echo "Presencia de patrón TTCC detectada"
        echo "Número de ocurrencias: $ttcc_count"
        echo "Nombre de secuencia: $name_sec"
    else
        echo "patrón TTCC no detectado"
    fi
done

```

```
fgalindom@U-1K50FU0JT4NHU:~/Documents/programacion_shell/galindo_k.pneumoniae_2026_nature/code$ bash analyze_sequences.sh
```

INFORME >Secuencia 1

Longitud de Secuencia: 135
Primer Nucleótido: G
Ultimo Nucleótido: A
Porcentaje GC (%GC): 49.63 %

DETECCIÓN DE PATRÓN GGGG

patron GGGG no detectado

DETECCIÓN DE PATRÓN TTCC

Presencia de patron TTCC detectada
Número de ocurrencias: 1
Nombre de secuencia: >Secuencia 1

INFORME >Secuencia 2

Longitud de Secuencia: 134
Primer Nucleótido: T
Ultimo Nucleótido: G
Porcentaje GC (%GC): 50.75 %

DETECCIÓN DE PATRÓN GGGG

patron GGGG no detectado

DETECCIÓN DE PATRÓN TTCC

Presencia de patron TTCC detectada
Número de ocurrencias: 1
Nombre de secuencia: >Secuencia 2

INFORME >Secuencia 3

Longitud de Secuencia: 134
Primer Nucleótido: A
Ultimo Nucleótido: T
Porcentaje GC (%GC): 47.76 %

DETECCIÓN DE PATRÓN GGGG

patron GGGG no detectado

DETECCIÓN DE PATRÓN TTCC

Presencia de patron TTCC detectada
Número de ocurrencias: 3
Nombre de secuencia: >Secuencia 3

INFORME >Secuencia 4

Longitud de Secuencia: 120
Primer Nucleótido: C
Ultimo Nucleótido: C
Porcentaje GC (%GC): 54.17 %

```
DETECCIÓN DE PATRÓN GGGG
-----
Presencia de patron GGGG detectada
Número de ocurrencias: 1
Nombre de secuencia: >Secuencia 4
Secuencia modificada: CCTCTGTAGTCAGCCCAATGGCGGTGGAGTCGCTCATGCTGGGCTAGTCAGATGTGGTCCGTTGATTCCACACGTTTCAGATGGCAGTCGTCAGATTCCTCACTATTGAAT"Mutation"ATC

DETECCIÓN DE PATRÓN TTCC
-----
Presencia de patron TTCC detectada
Número de ocurrencias: 2
Nombre de secuencia: >Secuencia 4

INFORME >Secuencia 5
-----
Longitud de Secuencia: 120
Primer Nucleótido: C
Ultimo Nucleótido: C
Porcentaje GC (%GC): 54.17 %

DETECCIÓN DE PATRÓN GGGG
-----
Presencia de patron GGGG detectada
Número de ocurrencias: 1
Nombre de secuencia: >Secuencia 5
Secuencia modificada: CCTCTGTAGTCAGCCCAATGGCGGTGGAGTCGCTCATTCTGGGCTAGTCAGGTGTGGTCCGTTGATTCCACACGTTTCAGATGGCAGTCGTCAGATTCCTCACTATTGAAT"Mutation"ATC

DETECCIÓN DE PATRÓN TTCC
-----
Presencia de patron TTCC detectada
Número de ocurrencias: 2
Nombre de secuencia: >Secuencia 5
```

8. Para concluir el proyecto, genere una copia de esta actividad cumplimentada en formato PDF y trasládelala al directorio **submission**. Además, incluya una captura de pantalla con la estructura de directorios actual usando el comando **tree** desde su directorio principal creado **User_Proyect_Year_Publication (0,25 pts)**.

Código:

Explicación:

Screenshot: